Software Testing

Aimed at testing game functional and implementation requirements – the exact same requirements outlined in your SRS. This section is the same as the Software stream (although it is assumed you will have far less things to test):

1. **Functional Test plan** – aimed at testing ***Functional requirements*** including testing strategy (white/black box) and types (unit, system, penetration, regression, etc), testing tools and resources assigned. This test plan should also include a detailed test schedule (with calendar dates, not university weeks), featuring testing milestones and test deliverables, covering scheduling and resourcing of all testing processes.

2. **Test case specifications** – For each test case specification, if a test has been completed include your latest results (i.e. actual result/output, pass/fail, etc); we would like the state of these to reflect that testing is ongoing. This test-case specification should include the following:
    a) Identifier(s)
    b) Test description.
    c) Input specifications (e.g. actual input to be tested or link to be tested - include range of test values i.e. possible valid and invalid input)
    d) Output specifications (e.g. expected outputs)

Test Specification – Part B: Test Case Specifications

ABC Fashion AI Shopping Agent MVP | Updated 19 October 2026

Purpose and interpretation of results

This document records the detailed inputs, expected behaviour and observed outcomes for functional, AI recommendation, preference extraction and preference validation tests. The original cases and their reported results are retained from the supplied Test Case Specifications PDF, with wording cleaned for readability and additional repository-driven regression coverage added. A recorded historical Pass means the supplied source reported that result; it does not mean the case was rerun during this document update.

Repository test evidence must be interpreted carefully. A recorded offline run on 19 October 2026 at 05:36:45 UTC reports 41/41 regression scripts passing. The latest execution attempt, at 13:05:29 UTC, was blocked before meaningful execution because `tsx` could not be loaded from `node_modules`; only the first script attempted to start and 0/41 scripts completed. Therefore the new repository regression table separates the last recorded successful results from the latest run status. The offline runner excludes live stock, browser journeys, security audit and client acceptance testing.

1. Original functional and API test cases

The following cases are the original test inventory from the supplied PDF. Product Details and cart cases cover the original commerce interface; AI cases cover catalogue retrieval, filtering/ranking and API validation; preference extraction and validation cases cover the structured customer brief. The historical Actual Result column is preserved where the PDF provides evidence, while cases without execution evidence remain Not Run.

Test Case ID	Test Description	Input Specification	Expected Output	Actual Result / Output	Pass / Fail / Not Run
TC-PD-01	Product Details page loads correctly(UC05)	Select a product from the catalogue	Page displays product image(s), name, price, colours and sizes from the catalogue	Product Details page loaded and displayed the product image, name, price, colour and size options as expected	Pass

TC-PD-02	Colour selection (UC05)	Click a different colour swatch	Selected colour is highlighted and product display updates accordingly	Historical note: the tested product had only one colour variant, so the interaction was not exercised. Re-test when multiple variants are available.	Not Run
TC-PD-03	Size selection (UC05)	Click an available size	Selected size is highlighted	Selected size was highlighted as expected	Pass
TC-PD-04	Add to Cart with valid selection (UC07)	Select colour and size, click Add to Cart	Item is added to the cart with the selected colour and size	Item was added to the cart with the selected colour and size	Pass
TC-PD-05	Add to Cart without size (invalid) (UC07)	Click Add to Cart without selecting a size	Item is not added; user is prompted to select a size	Item was not added to the cart and the user was prompted to select a size	Pass
TC-PD-06	Product description section (UC05)	Click the description section, then click again	Section expands to show details; clicking again collapses it	Section expanded and collapsed as expected	Pass
TC-PD-07	Mobile layout (UC05)	Open the Product Details page on a mobile browser	Layout adjusts to the screen without overlapping or cut-off content	Layout displayed correctly on a mobile browser with no overlapping or cut-off content	Pass

TC-PD-08	Add same item to cart with different size (UC07)	Select a product and size S, then add it to the cart. Then, select size M, and add it again	The cart shows two separate rows for the same product, one for size S and one for size M. The Order Summary update to reflect both items.	Two items were added in separate rows in the cart. The Shopping Cart number and Order Summary updated correctly	Pass
TC-PD-09	Add same item to cart twice (UC07)	Select the same product and size, then click Add to Cart twice.	Its quantity increases to 2. The Shopping Cart number and Order Summary update.	The quantity increased to 2, and the Shopping Cart number and Order Summary updated correctly.	Pass
TC-PD-10	Remove item from cart (UC07)	Click Remove on a cart item. .	The item disappears. The Shopping Cart number and Order Summary recalculate.	The item was removed, and the Shopping Cart number and Order Summary updated correctly.	Pass
TC-PD-11	Adjust item quantity using the + and − buttons (UC07)	Click + , then − , on a cart item.	cart item.The quantity changes by one with each click. The Shopping Cart number and Order Summary update accordingly.	The quantity changed as selected, and the Shopping Cart number and Order Summary updated correctly.	Pass
TC-PD-12	Click to Products in cart (UC07)	Click a product in the cart	The corresponding product details page opens.	The correct product details page opened.	Pass

TC-PD-13	Click to Continue Browsing Collection on cart page (UC07)	Click Continue Browsing Collection.	The product browsing page opens.	The product browsing page opened	Pass
TC-PD-14	Click to Need personal styling advice? Contact Customer Concierge on cart page (UC07)	Click the styling advice link.	The linked styling advice page opens.	The linked styling advice page opened.	Pass
TC-AI-01	Product catalogue retrieval	Request catalogue products from Supabase	System retrieves available catalogue products successfully	System retrieved 316 products from the Supabase catalogue	Pass
TC-AI-02	Hard filtering by size and budget	Size = 10, max price = $100	Only available, in-stock products containing size 10 and costing ≤ $100 are returned	60 products remained after filtering; returned samples all contained size 10 and were ≤ $100	Pass
TC-AI-03	Soft preference ranking	Size 10, max $100, category Clothing, preferred colour Lavender	Products passing hard constraints are ranked by preference match	3 exact matches were returned; all matched Clothing + Lavender with score 2/2	Pass

TC-AI-04	Full recommendation pipeline	“Show me lavender clothing for a beach wedding, with a relaxed style, in size 10, costing no more than $100 per item.”	Preferences are extracted, products filtered/ranked, and best recommendation returned	Returned Lavender Co-ord Set (P1002) at $55 with size 10 available and	Pass
TC-AI-05	Maximum budget boundary	Lavender clothing, size 10, maximum price $75	Products above $75 are excluded; products at exactly $75 remain valid	$55 Co-ord Set and $75 Cardigan returned; $100 Pleated Skirt excluded	Pass
TC-AI-06	Fallback alternatives when no exact match	Lavender clothing, size 10, max price $50	Hard constraints remain enforced; soft preferences may be relaxed and unmatched preferences identified	Alternatives under $50 were returned; each matched Clothing but clearly reported Lavender as unmatched	Pass
TC-AI-07	Explicit colour exclusion	Lavender clothing, size 10, max $50, exclude Brown	Brown products must not be returned, even as alternatives	Brown Linen Shirt was removed; only non Brown alternatives were returned	Pass
TC-AI-08	Required size enforcement	Lavender clothing, size 12, max $50	Products not available in size 12 must be excluded	Brown Linen Shirt, which listed only sizes 6, 8 and 10, was not returned; all returned products listed size 12	Pass

TC-AI-09	Occasion and style ranking	Clothing, Lavender, Beach Wedding, Relaxed, size 10, max $100	Product matching all four soft preferences should rank as an exact match	P1002 Lavender Co-ord Set returned with 4/4 matching Category,	Pass
TC-AI-10	No product satisfies hard constraints	Size 10, max price $1	No unsuitable or over budget product should be returned; response should indicate no match	Filter returned 0 eligible products and recommendation API returned no_match with an empty product list	Pass
TC-AI-11	Empty/invalid shopping request handling	Blank or missing shopping message	API rejects invalid input without processing recommendations	Blank and missing messages were rejected with HTTP 400 during API validation testing	Pass
TC-AI-12	Invalid recommendation limit	Recommendation request with limit = 7	API rejects values above the supported limit of 6	Request was rejected with HTTP 400	Pass
TC-AI-13	Malformed JSON request	Invalid JSON body	API returns controlled validation error instead of crashing	Malformed JSON was rejected with HTTP 400	Pass
TC-AI-14	Incorrect content type	Send request using text/plain instead of application/json	API rejects unsupported content type	Request was rejected with HTTP 415	Pass

TC-AI-15	AI service unavailable	Simulate OpenAI/API failure	System should return an understandable error without exposing sensitive details	Not yet tested; failure-path simulation has not been evidenced in the supplied repository logs.	Not Run
TC-AI-16	Database unavailable	Simulate Supabase/database failure	System should return an appropriate catalogue unavailable error without returning fabricated product results.	Not yet tested; database-outage simulation has not been evidenced in the supplied repository logs.	Not Run
TC-PE-01	Extract dress, occasion, colour, size and budget	“I need a black dress under $150 for a wedding in size 10.”	Structured preferences identify Clothing, Dresses, Wedding Guest, Black, size 10 and maximum price $150	All specified values were extracted correctly; unspecified fields remained null or empty lists	Pass
TC-PE-02	Avoid inferring an unstated product category	“I’m going to a beach wedding and want something elegant, maybe sage or blue.”	Extract Beach Wedding, Elegant, Sage and Blue without assuming a dress, category, gender or season	Occasion, style and colours were extracted; category, subcategory, product type, gender and season remained null	Pass
TC-PE-03	Extract a casual brunch request	“I need something casual for brunch under $100.”	Extract Brunch, Casual and maximum price $100 without assuming a product category	Returned occasion Brunch, style Casual and maximum price $100; category remained null	Pass

TC-PE-04	Extract gender, category and size	“Show me men’s shoes for work in size 11.”	Extract Men, Shoes, Work and size 11 without inventing a shoe subcategory	Returned Men, Shoes, Work and size 11; subcategory remained null	Pass
TC-PE-05	Extract an excluded colour	“I want a romantic outfit for date night but nothing red.”	Extract Romantic and Date Night; put Red in excluded colours, not preferred colours	Returned Romantic, Date Night, excluded colours ["Red"] and an empty preferred-colours list	Pass
TC-PE-06	Extract minimum and maximum prices	“A bag costing at least $100 and up to $150.”	Extract Bags, minimum price $100 and maximum price $150	Returned Bags with min_price: 100 and max_price: 150	Pass
TC-PE-07	Reject empty extraction input	Pass a whitespace-only message to extractPreferences()	Function rejects the request before calling OpenAI	Function rejected the blank message with “Customer message cannot be empty”	Pass
TC-PE-08	Reject invalid extraction API bodies	Send malformed JSON, a missing message, a non-string message or a whitespace-only message to POST /api/extraction	API returns HTTP 400 with a controlled validation error	Each invalid body returned HTTP 400 during API testing	Pass
TC-PV-01	Normalise recognised catalogue values	Style “ elegant ” and preferred colours [" navy ", "NAVY"]	Return catalogue spelling Elegant and one Navy, without changing the original input	Returned Elegant and ["Navy"]; the input object was unchanged	Pass

TC-PV-02	Flag an unsupported soft preference	Occasion Brunch and style Casual	Preserve both values and report Casual as an unsupported catalogue style without blocking the request	Brunch and Casual were preserved; a non-blocking style issue was returned	Pass
TC-PV-03	Flag an unknown size	Size “99”	Preserve the requested size, report it as unsupported and require clarification	Returned a blocking size issue and needs_clarification: true	Pass
TC-PV-04	Flag an unknown excluded colour	Exclude “Ultraviolet”	Preserve the exclusion and require clarification so retrieval cannot silently ignore it	Returned a blocking excluded-colour issue and needs_clarification: true	Pass
TC-PV-05	Detect a category mismatch	Category Shoes with subcategory Dresses	Report that Dresses does not belong to Shoes and require clarification	Returned a category-mismatch issue and needs_clarification: true	Pass
TC-PV-06	Detect a conflicting budget	Minimum price $200 and maximum price $100	Report the invalid price range and require clarification	Returned an invalid-budget issue and needs_clarification: true	Pass
TC-PV-07	Validate against the live Supabase vocabulary	Occasion Brunch and style Casual, using the public catalogue connection	Recognise Brunch and flag Casual as an unsupported style	Read 316 publicly visible products; recognised Brunch and returned a non-blocking issue for Casual	Pass

2. Repository offline regression suite

The repository regression runner (`npm test`) defines 41 offline scripts. The saved regression log dated 19 October 2026 at 05:36:45 UTC records all 41 scripts passing. Each row below describes the script's intended coverage, the action or execution method, the expected outcome and the status recorded in that log. "Pass (recorded)" refers to that saved run.

Test Case ID	Test Script	Test Description	Input / Action	Expected Output	Status
TC-REG-01	test-stylist-remediation.mjs	Checks that the stylist can recover from an incomplete or invalid recommendation state and provide a useful corrective response.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-02	test-conversation-recovery.mjs	Checks that a conversation can recover from an interrupted, incomplete or invalid turn while retaining relevant context.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-03	test-matching-set-budget.mjs	Checks that a matching set is assembled within the requested budget and that price constraints are respected.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-04	test-category-collection-defaults.mjs	Checks default category and collection handling when the customer gives a broad or partially specified shopping request.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-05	test-collection-first.mjs	Checks that collection discovery is prioritised when the customer begins with a broad browsing request.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-06	test-product-card-presentation.mjs	Checks that recommended products are presented in the expected product-card structure with key product details.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-07	test-guided-discovery.mjs	Checks the guided-discovery flow for eliciting missing shopping preferences and moving the customer toward suitable products.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-08	test-search-filters.mjs	Checks that search applies the supported filters and respects the customer’s specified shopping constraints.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-09	test-selected-outfit-review.mjs	Checks that a selected outfit can be reviewed and that its displayed items and summary are represented consistently.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-10	test-outfit-interface.mjs	Checks outfit-interface behaviour, including the expected presentation and interaction with outfit items.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-11	test-grouped-discovery-memory.mjs	Checks that grouped product discovery retains relevant memory/context across related discovery turns.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-12	test-outfit-budget.mjs	Checks that outfit-level pricing is calculated and recommendations respect the requested total budget.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-13	test-customer-scenario-regressions.mjs	Runs regression assertions for representative customer shopping scenarios and expected conversational outcomes.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-14	test-preference-validation.ts	Checks validation of extracted shopping preferences, including unsupported or inconsistent values.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-15	test-recommendation-contract.ts	Checks that recommendation results conform to the expected structured response contract.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-16	test-grounded-recommendation.ts	Checks that recommendations are grounded in available catalogue data rather than unsupported or fabricated product details.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-17	test-validation-search.mjs	Checks the interaction between preference validation and product search, including handling of invalid or unsupported criteria.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-18	test-guided-follow-ups.mjs	Checks follow-up prompts that clarify missing details and refine recommendations using the customer's answers.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-19	test-cross-sell.mjs	Checks cross-sell recommendation logic for relevant complementary products.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-20	test-cross-sell-client.mjs	Checks client-side handling and presentation of cross-sell recommendations.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-21	test-upsell.mjs	Checks upsell logic for relevant higher-value alternatives while respecting the request context.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-22	test-upsell-client.mjs	Checks client-side handling and presentation of upsell recommendations.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-23	test-chat-request.mjs	Checks construction and handling of chat requests, including the expected request fields and response behaviour.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-24	test-optional-discovery.mjs	Checks that optional discovery questions do not block progress when the customer has provided enough information to search.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-25	test-card-actions.mjs	Checks the supported actions on product cards and that each action is handled correctly.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-26	test-cheaper-search.mjs	Checks that a request for cheaper options triggers a revised search and returns lower-priced eligible alternatives.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-27	test-viewed-product.mjs	Checks that the conversation can use the currently viewed product as context for subsequent recommendations or questions.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-28	test-conversation-foundation.mjs	Checks foundational conversation behaviour, including initial state and basic message processing.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-29	test-conversation-plan.mjs	Checks creation and handling of the shopping plan derived from the customer's brief.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-30	test-conversation-search.mjs	Checks that conversation intent and retained preferences are passed into product search appropriately.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-31	test-plan-evidence.mjs	Checks that shopping-plan decisions are supported by the available plan or recommendation evidence.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-32	test-plan-recovery.mjs	Checks recovery of a shopping plan when plan state or the previous step is incomplete or needs correction.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-33	test-conversation-answers.mjs	Checks answers to customer questions in conversation and whether responses use the available shopping context.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-34	test-conversation-production.mjs	Checks production-oriented conversation behaviour and expected handling of a normal shopping interaction.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-35	test-stylist-commerce.mjs	Checks commerce-related stylist behaviour, connecting recommendation flows with shopping actions and product information.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-36	test-stylist-feature-import.mjs	Checks that stylist feature modules/imports load and expose the functionality required by the application.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-37	test-product-specifications.mjs	Checks product specification handling and representation, including the expected product attributes.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-38	test-stylist-telemetry.mjs	Checks stylist telemetry/event reporting for expected events without disrupting the user-facing flow.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-39	test-conversation-integration.mjs	Checks integration between conversation handling and the supporting recommendation/search components.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)
TC-REG-40	test-shopping-presentation.mjs	Checks presentation of shopping results so the recommendation output is readable and structured for customers.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

TC-REG-41	test-friendly-shopping-message.mjs	Checks that shopping messages use clear, customer-friendly wording for the relevant response state.	Offline automated regression script	Script assertions pass with exit code 0.	Pass (recorded)

3. Additional repository-driven customer scenario evidence

The repository contains a customer-scenario assessment dated 17 October 2026. It evaluates 20 customer scenarios with repeated runs and is more representative of multi-turn shopping than isolated component tests. The report explicitly states that component passes do not constitute browser acceptance, and that the application/production build failed during that assessment. The aggregate results below are copied from that report and should be treated as historical evidence until a new assessment is run against a working build.

Test Case ID	Scenario / test layer	Expected outcome	Recorded result	Status / follow-up
TC-CS-01	20-case real-model customer scenario assessment, 3 runs per case (60 runs)	Customer journeys preserve explicit constraints, resolve follow-up edits, return grounded products and complete the intended shopping task.	23 Pass, 33 Fail, 1 Blocked, 3 Feature gap across 60 recorded case runs; 121 turn checks passed, 53 failed and 15 dependent checks were blocked/excluded.	Fail / incomplete. Historical assessment dated 17 Oct 2026; rerun after restoring app/build.

TC-CS-02	Hard-constraint enforcement in conversation	Explicit colour, size, occasion and budget requirements must not be omitted or reclassified by the interpreter.	Report identifies black-bag requests where Black was omitted or misclassified as style, allowing beige product B-OPEN-45 to be returned.	Fail in historical assessment. Add raw-brief constraint assertions and retest.
TC-CS-03	Multi-turn refinement of displayed shortlist	Simple colour, budget and style edits should update the current shopping scope without unnecessary product-selection clarification.	Report records repeated “Which item?” or generic clarification responses for ordinary refinements.	Fail in historical assessment. Retest blanket refinements against working app.
TC-CS-04	Category, product type and product reference resolution	Broad item names should map to the correct catalogue hierarchy and visible products.	Report records Bags treated as an exact subcategory and Necklace mapped to an incorrect hierarchy field in some runs.	Fail in historical assessment. Retest category defaults and product reference resolution.

TC-CS-05	Outfit state and ownership transitions	Likes must not silently add/own an item; owned items must remain owned; duplicate operations should be resolved safely.	Report records an “I like the first dress” turn mutating outfit state and an own/keep conflict for one product.	Fail in historical assessment. Retest state transitions and persistence.
TC-CS-06	Build and browser/live endpoint smoke test	App starts, production build succeeds and live endpoint returns valid structured responses.	Report states TypeScript/build failed due to missing `@vercel/analytics/next`; homepage/live conversation returned HTTP 500.	Fail / blocked in historical assessment. Recheck current dependency manifest and clean build.
TC-CS-07	Browser UI, real cart, persistence, images and aesthetic judgement	End-to-end UI and persisted user-visible behaviour are verified in a browser.	Historical report explicitly says these behaviours were unverified because the app did not start correctly.	Not Run / unverified. Manual browser evidence required.

4. Outstanding tests and required retesting

The following work is required to close the principal evidence gaps. A test should only be marked Pass after the exact input has been executed against the intended build and the output or supporting artifact has been checked.

Test Case ID	Test description	Input / action	Expected output	Actual result / output	Status
TC-ENV-01	Restore regression test environment	Perform a clean, reproducible dependency install in the repository and run `npm test`.	All 41 regression scripts complete; each result and environment is saved to a new log.	Latest attempt stopped because `tsx` was missing; 0/41 scripts completed.	Blocked / retest required
TC-INT-01	OpenAI service unavailable	Simulate a controlled OpenAI API failure using a safe test configuration.	Return a clear controlled error; do not leak credentials or fabricated recommendations.	No completed failure-injection evidence found in supplied logs.	Not Run
TC-INT-02	Supabase/database unavailable	Simulate a database connection/query failure.	Return a controlled catalogue-unavailable response; do not fabricate products.	No completed failure-injection evidence found in supplied logs.	Not Run
TC-UI-01	Colour variants on Product Details	Choose a product with at least two real catalogue colour variants and switch swatches.	Selected swatch and displayed product variant update consistently.	Original test was not exercised because the tested product had only one colour option.	Not Run

TC-UI-02	Responsive Product Details and cart	Run product selection, add/remove, quantity and order summary flows on desktop and mobile browsers.	No overlap/cut-off; cart count, quantities and order summary stay consistent.	Historical manual passes are present in the source PDF; no fresh browser execution is evidenced in the latest regression log.	Historical Pass; fresh retest Not Run
TC-INT-03	Live catalogue count and stock grounding	Query live Supabase catalogue and compare returned product IDs, prices, sizes and stock with source rows.	Only real eligible catalogue products are returned; values match the current database.	Historical evidence references 316 products; current count must be reverified.	Retest required
TC-ACC-01	Client acceptance test	Run agreed end-to-end scenarios on the deployed build with client representative.	Client confirms MVP meets agreed SRS requirements and any defects are recorded.	No acceptance result evidenced in the supplied repository logs.	Not Run

5. Execution rules and evidence recording

For each fresh execution, record tester/executor, date and time, commit or build identifier, environment, input, expected result, actual result, status and evidence path.

Use Pass only when actual output meets the expected output. Use Fail for a functional mismatch. Use Blocked when setup, credentials, unavailable services or another prerequisite prevents a valid execution. Use Not Run when the case has not been attempted.

Keep offline deterministic test results, live API tests, browser acceptance and client acceptance separate. Do not generalise a 41/41 offline pass to production readiness.

Attach screenshots for UI states, request/response excerpts for API cases, timestamped logs for regression cases, and relevant transcript or JSON evidence for AI conversation scenarios. Redact secrets and personal information.

After fixes, rerun the failing test and related regression cases; update the status and preserve the earlier result for traceability.

Repository evidence referenced in this revision: `docs/test-evidence/regression-2026-10-19T05-36-45-919Z.txt` (41/41 historical pass); `docs/test-evidence/regression-2026-10-19T13-05-29-788Z.txt` (latest blocked attempt); `docs/ai-stylist-customer-scenarios-report-2026-10-17.md` (customer scenario assessment and limitations).